BAIL security

SECURITY AUDIT

Lista Dao

Yield Buffer

FINAL REPORT

June 2026

BAILSEC.IO

Disclaimer:

Security assessment projects are time-boxed and reliant on information provided by the client, its affiliates, or its partners. The findings in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase, and apply only to the specific commit hash, version, or deployment reviewed at the time of the audit. Any subsequent modification, redeployment, or upgrade is outside the scope of this report and may invalidate its findings.

The audit does not constitute investment, legal, financial, regulatory, or tax advice, nor an endorsement of any project or team. It is based on a limited review of materials provided and is delivered on an "as-is," "where-is," and "as-available" basis. Use of blockchain technology is subject to unknown risks and flaws. The scope of this assessment is technical and contract-level; unless explicitly stated otherwise, it does not cover economic or game-theoretic risks, MEV, oracle manipulation, governance attacks, centralization or admin-key risks, off-chain components, front-end code, or the behavior of third-party protocols, dependencies, libraries, or services interacted with by the audited code.

We make no warranties, express or implied, regarding the accuracy, reliability, completeness, or availability of the report or any associated services, and disclaim all implied warranties including merchantability, fitness for a particular purpose, and non-infringement. We assume no responsibility for any product, service, software, code, or material referenced by, linked to, or accessible through the report.

This report is prepared solely for the contract owner and may not be relied upon by any third party. The contract owner is responsible for making their own decisions based on the report and should seek additional professional advice as needed. To the maximum extent permitted by law, our total aggregate liability arising from or related to this audit shall not exceed the fees actually paid for the engagement. The contract owner agrees to indemnify and hold harmless the audit firm and its personnel from any claims, damages, expenses, or liabilities arising from use of or reliance on the report or the audited smart contract.

This disclaimer and any dispute arising from the audit shall be governed by the laws of Wyoming (USA), without regard to its conflict-of-laws principles. By engaging in this audit, the contract owner acknowledges and agrees to these terms.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Lista Dao – Yield Buffer – Audit Report
Website	Lista.org
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/lista-dao/moolah/tree/47d3fc274286b947fd9f7d6778e83e0a29944c80
Resolution 1	https://github.com/lista-dao/moolah/tree/5a2dc56c59225d9d818f88f33aaec30709d8b284

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High	1	1				
Medium	6	2		4		
Low	11	2		9		
Informational	11	4		6		1
Governance	1			1		
Total	30	9		20		1

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

Global

Issue_01	Governance Privilege
Severity	**Governance**
Description	Currently, the contract exposes one or more governance privileges which can result in unexpected operations and potential loss of funds. For example via a proxy implementation upgrade
Recommendations	Consider implementing a strong multisig setup for all privileged addresses in the system.
Comments / Resolution	Acknowledged.

Issue_02	Lack of support for transfer tax tokens
Severity	**Informational**
Description	The current architecture is not meant to be used with transfer-tax tokens. If such tokens would ever be used, this would break the accounting which results in multiple critical side-effects.
Recommendations	Consider not using such tokens.
Comments / Resolution	Acknowledged.

BrokerInterestRelayer

The BrokerInterestRelayer contract is a simple relayer contract which relays the interest which is accrued via the LendingBroker, to the MoolahVault.

The following changes have been made:

- implementation of notifyBrokerInterest call

The diff can be found here:

- https://www.diffchecker.com/5g2AgI2q/

Invariants

INV 1: Each relay must call notifyBrokerInterest if the vault has a corresponding buffer.

No issues found.

CreditBrokerInterestRelayer

The CreditBrokerInterestRelayer contract is a simple relayer contract which relays the interest which is accrued via the CreditBroker, to the MoolahVault.

The following changes have been made:

- implementation of notifyBrokerInterest call

The diff can be found here:

- https://www.diffchecker.com/L9yHMtBd/

Invariants

INV 1: Each relay must call notifyBrokerInterest if the vault has a corresponding buffer.

No issues found.

MoolahVault

The MoolahVault contract is an ERC4626 style contract which allows users to deposit assets and these assets are then allocated to one or multiple Moolah markets. It contains various different protocol-unique specifics but these are not part of this review.

The following changes have been made:
- implementation of _name and _symbol variables
- implementation of lockBuffer variable
- removal of ROLE_ADMIN for CURATOR
- implementation of setLockBuffer()
- implementation of setWhitelist()
- implementation of isWhitelist()
- implementation of getWhitelist()
- initProvider function changed to setProvider
- implementation of setName()
- implementation of setSymbol()
- protection of mint and deposit with whitelist
- Incorporation of Buffer.currentLocked into totalAssets()
- implementation of name()
- implementation of symbol()
- implementation of setRoleAdmin()

The diff can be found here:
- https://www.diffchecker.com/nY9ef5Zv/

Privileged Functions

- setLockBuffer
- setWhitelist
- setName
- setSymbol

Invariants

INV 1: Only whitelisted addresses can deposit/mint

INV 2: Buffer.currentLocked must decrease current available assets

Issue_03	DoS of multiple broker paths due to edge-case in notification check
Severity	High
Description	The new broker-yield smoothing mechanism routes broker revenue through dedicated relayers. When a broker repayment, penalty, conversion, liquidation, or credit-broker recovery produces vault revenue, the relayer eventually supplies the accumulated amount into Moolah on behalf of MoolahVault. After that supply, the relayer reads MoolahVault.lockBuffer() and calls BrokerInterestLockBuffer.notifyBrokerInterest(amount). The buffer records the notified amount as lockedAmount, and MoolahVault.totalAssets() subtracts currentLocked() from the vault's counted Moolah supply assets. This makes the broker revenue visible gradually instead of immediately. The critical interaction is that notifyBrokerInterest() itself calls IERC4626(vault).totalAssets() before recording the new notification. Therefore, the notification validation is performed against the vault's already-smoothed totalAssets(), not against raw Moolah assets and not against the actual increase caused by the relayer's MOOLAH.supply() call. BrokerInterestLockBuffer.notifyBrokerInterest(amount) rejects the notification if amount > IERC4626(vault).totalAssets(). At the point of this check, the relayer has already supplied the new broker revenue into Moolah on behalf of the vault, but vault.totalAssets() still subtracts the old active lock. Let: • R = vault's raw counted Moolah assets before the new relayer supply; • L = existing currentLocked() before the new notification; • A = new broker revenue amount supplied and notified. If the relayer supply is fully counted by the vault, raw counted assets

become:

R + A

However, vault.totalAssets() reports:

max(R + A - L, 0)

The buffer requires:

A <= max(R + A - L, 0)

This condition fails whenever:

L > R

So if the existing locked amount is greater than the vault's pre-flush counted assets, any positive threshold-crossing broker notification can revert, even if the new broker revenue was supplied into Moolah correctly.

This state is reachable. The vault can enter it **after Moolah bad debt reduces market.totalSupplyAssets**, after market removal causes assets to stop being counted by withdrawQueue, or after an over-lock state where the buffer previously locked more than the actual vault-counted NAV increase.

Bad debt socialization setup to reach **Initial state:**

Assume two users in the Moolah market: Moolah Vault (MV) and Alice (AL)

- Global State - totalSupplyAssets = 1000e18 , totalBorrowAssets = 143.5e18 , totalBorrowShares = 143.5e18, totalSupplyShares = 1000e18
- MV State - supplyShares = 1000e18 (assume share price = 1.0, hence sole supplier)
- AL State - collateral = 10e18 (worth 50e18 assets at price = 5e18), borrowShares = 143.5e18 (= 143.5e18 assets of debt, sole borrower)
- At this point, Alice is already insolvent (assuming a price crash)

since collateral is worth 50e18 against debt of 143.5e18.

Liquidation flow:

- Assume liquidator seizes all collateral
- seizedAssetsQuoted = 10e18 × 5e18 / 1e18 = 50e18
- repaidShares = ceil(50e18 / 1.15) = 43.478e18
- repaidAssets = 43.479e18
- totalBorrowAssets = 143.5e18 – 43.479e18 = 100.021e18
- totalBorrowShares = 143.5e18 – 43.478e18 = 100.022e18
  Alice's borrowShares = 143.5e18 – 43.478e18 = 100.022e18

Since Alice's collateral = 0, bad debt block is triggered.

- badDebtShares = 100.022e18
- badDebtAssets = min(100.021e18, ceil(100.022e18 × 100.021e18 / 100.022e18)) = min(100.021e18, ceil(99.999...e18)) = 100e18
- totalBorrowAssets = 100.021e18 – 100e18 = 0.021e18
- totalBorrowShares = 100.022e18 – 100.022e18 = 0
- totalSupplyAssets = 1000e18 – 100e18 = 900e18 (the loss)

Initial state (can occur due to bad-debt socialization as described in above example):

- Raw counted vault assets before new broker revenue: R = 900
- Existing active buffer lock: L = 1,000
- New broker revenue amount: A = 100

The relayer supplies 100 into Moolah on behalf of the vault.

Raw counted vault assets become:

900 + 100 = 1,000

Before recording the new notification, the buffer checks vault.totalAssets().

The vault subtracts the existing lock:

	max(1,000 - 1,000, 0) = 0 The buffer then checks: 100 > 0 The check fails and notifyBrokerInterest(100) reverts. Because this revert happens inside the relayer call, the whole upstream repayment, liquidation, conversion, or credit-broker recovery transaction reverts. Impact 1: Legitimate broker revenue flushes can become unavailable while the vault is in a lock-deficit state. Impact 2: Borrower repayment flows that include interest can revert once the relayer crosses minLoan. Impact 3: Liquidation paths that produce broker interest can revert at the relayer notification step, creating liquidation liveness risk during stressed market conditions. Impact 4: Credit-broker repayments, penalties, and bad-debt recoveries routed through CreditBrokerInterestRelayer can become blocked at the threshold-crossing repayment. Impact 5: The issue can persist until the existing lock decays below raw counted

	vault assets, counted assets recover, or the buffer configuration changes. Note that this will DoS all shared brokers.
Recommendations	Consider removing the check and ensure that the vault is always sufficiently seeded to prevent scenarios where the notified amount is larger than the totalAssets value.
Comments / Resolution	Resolved, the check has been removed.

Issue_04	Side-effects due to zero totalAssets
Severity	**Medium**
Description	MoolahVault reports its NAV as totalAssets() = ΣexpectedSupplyAssets(withdrawQueue) – currentLocked(), floored at zero, so that brokered interest just supplied to a Moolah market is hidden by the buffer's currentLocked and revealed linearly over duration. This single value drives every value-sensitive path: _convertToShares/Assets and the deposit/withdraw/redeem pricing, _accruedFeeShares fee accrual, _maxWithdraw, and the buffer's own notifyBrokerInterest cap. The vault supplies LP liquidity into Moolah markets that the brokers borrow from; when a borrower goes bad, the loss is socialized in the Moolah market (a slash), reducing every supplier's, including the vault's expectedSupplyAssets. The interaction of a loss in raw with a still-active, time-only currentLocked is the crux. currentLocked() is a function of lockedAmount and elapsed time only; it is never reduced in response to a drop in raw, and totalAssets() unconditionally subtracts it whenever raw > locked. A slash reduces raw by the loss D. Normally that should leave totalAssets ≈ raw. But because the buffer keeps subtracting the full currentLocked, once D is large enough that raw ≤ currentLocked, totalAssets() is clamped to 0, the lock over-subtracts the entire remaining real position. The protocol assumed the lock would always be backed by at least currentLocked of counted assets (true at notify time, since the supply precedes the notify), but a subsequent slash breaks that assumption and nothing re-syncs the lock. The boundary is reachable through the ordinary liquidation/bad-debt path; no check prevents totalAssets() from reading zero while the vault still holds real, slashed-down value.

	Impact 1: With totalAssets() == 0, deposit/mint price against a denominator of newTotalAssets + 1 = 1, minting a near-unbounded share count; an opportunist who deposits dust captures essentially all subsequent value, draining the vault's remaining (locked-backing) value from the already-slashed holders as currentLocked decays and the real position re-surfaces. Impact 2: The notify cap amount > IERC4626(vault).totalAssets() becomes amount > 0, so every interest-supplying broker flow, repay, penalty, and LendingBroker.liquidate, reverts. The liquidations needed to contain the slash's bad debt are therefore bricked exactly when they are most needed, a self-reinforcing insolvency loop. (Already raised in a separate issue) Impact 3: maxWithdraw/maxRedeem collapse and redeem prices to ~0 assets, so honest LPs cannot exit at any value for the duration of the clamp window; and _accruedFeeShares computes zero interest (zeroFloorSub), so the performance fee on the period is forgone while lastTotalAssets ratchets down.
Recommendations	While impact 2 can be solved as discussed in a separate issue, there is no general fix that does not require rewriting the logic fully. Since this is a very rare situation, we recommend observing markets and pause them until the buffer has expired. The pausing may have secondary impacts if relayers share markets.
Comments / Resolution	Acknowledged, the team will act accordingly in that unlikely situation

Issue_05	MoolahVault.permit() is incompatible with standard EIP-2612 signers; metadata changes affect integrations but not permit
Severity	Medium
Description	MoolahVault is an ERC4626 vault token that inherits ERC20PermitUpgradeable, intended to support EIP-2612 gasless approvals (permit) so integrators can approve-and-act in one signed message. EIP-2612/EIP-712 derives a DOMAIN_SEPARATOR from the token name, a version string, the chain id, and the verifying contract address; signatures are valid only against the exact domain the contract reconstructs. The vault also exposes admin-settable display metadata: name()/symbol() are overridden to return _name/_symbol, which setName/setSymbol (DEFAULT_ADMIN_ROLE) can change at any time. The permit domain and the display metadata are two distinct surfaces that integrators rely on. initialize wires up ERC4626 and ERC20 (__ERC4626_init, __ERC20_init) and access control, but never calls __ERC20Permit_init (which would call __EIP712_init with the token name). Because of that, the inherited EIP712Upgradeable state holding the domain name is never set, so _EIP712Name() returns the empty string and permit() builds its DOMAIN_SEPARATOR with keccak256(""). A standard EIP-2612 signer (a wallet/router/zap) constructs its signing domain from the token's name(), the actual, non-empty name, so the signature it produces is valid only against a domain with that name, while the contract verifies against the empty-name domain. The recovered signer therefore never matches and permit() reverts. The metadata mutability is a second, related surface: setName/setSymbol can change name() after deployment, which can desync integrations and displays that cached the original name, but it does not further affect permit, precisely because the permit domain is

	decoupled from name() and already fixed at the empty value. Nothing in the contract reconciles the two: the permit domain is built from the uninitialized EIP-712 name, never from the live name().
Recommendations	Consider adding documentation that explains that the permit domain is built from the uninitialized EIP-712 name. We do not recommend any other change because some vaults are already live and this would require a more complex upgrade structure with reinitializers which could introduce hidden side-effects.
Comments / Resolution	Acknowledged.

Issue_06	Orphaned buffered rewards if vault becomes empty
Severity	Medium
Description	The new broker-yield smoothing mechanism does not escrow rewards. When broker revenue is flushed through BrokerInterestRelayer or CreditBrokerInterestRelayer, the relayer first supplies the assets into Moolah on behalf of MoolahVault. The vault immediately receives real Moolah supply shares. The relayer then notifies BrokerInterestLockBuffer, which records the amount as lockedAmount. MoolahVault.totalAssets() sums the vault's Moolah supply assets and then subtracts BrokerInterestLockBuffer.currentLocked(). This makes the reward invisible to ERC4626 share pricing until the lock decays. The relevant accounting surfaces are therefore split. At the Moolah layer, the vault owns the reward immediately. At the ERC4626 layer, share holders only receive the reward gradually as currentLocked() decreases. The vault does not checkpoint which shares existed when the reward was generated. The root cause is that the buffer is global and time-based, not share-supply-aware. It keeps releasing value even if the share supply that existed when the reward was generated has fully exited. If all LPs redeem or withdraw while currentLocked() > 0, their shares are burned against the smoothed totalAssets() value, which excludes the locked reward. The raw Moolah supply shares corresponding to the locked reward can remain owned by the vault address. After totalSupply() reaches zero, the buffer continues decaying with time. Once currentLocked() decreases, the previously hidden Moolah assets become visible in MoolahVault.totalAssets(). At that point, the vault can have totalAssets() > 0 and totalSupply() == 0. The vault's conversion formula then treats this as a zero-supply vault with existing assets: shares = assets * (totalSupply + 10 ** decimalsOffset) / (totalAssets + 1)

	This means the orphaned assets are not cleanly distributed to the historical LPs that held shares when the broker reward was created. They also are not cleanly free value for the next depositor; the virtual-share / virtual-asset terms absorb the existing assets into the initial exchange rate and can make the value behave like a donation to virtual reserves. An additional side-effect is that this lays the path for the first-depositor attack as now a user can deposit, accrue rewards without donating and then future depositors receive less shares.
Recommendations	Consider executing a governance deposit which is never withdrawn.
Comments / Resolution	Resolved, governance will execute the first deposit.

Issue_07	Bad-Debt will overstate vault losses in case of remaining dripping interest
Severity	Medium
Description	BrokerInterestLockBuffer is designed to smooth brokered yield that is supplied into Moolah on behalf of MoolahVault. When a broker relayer flushes yield, it first calls MOOLAH.supply(..., vault, ""), which gives the vault real Moolah supply shares. It then calls notifyBrokerInterest(amount), which records a nominal locked amount in the buffer. MoolahVault.totalAssets() sums the vault's expected Moolah supply assets across withdrawQueue, then subtracts lockBuffer.currentLocked(). This means the vault owns the broker-yield Moolah shares immediately, but the corresponding ERC4626 NAV is hidden and released over time. Moolah bad debt reduces market[id].totalSupplyAssets. If the vault has supply shares in the affected market, its expected supply assets decrease. The buffer, however, is vault-global and nominal. It does not track which market backs the lock and does not reduce lockedAmount when the underlying Moolah supply assets are impaired. The root cause is that the buffer assumes the locked amount remains fully backed by raw Moolah assets until it decays linearly. That assumption fails when bad debt reduces the Moolah supply assets that include the locked broker yield. The loss is applied at the Moolah market level by reducing totalSupplyAssets. This immediately reduces the vault's raw counted assets. However, BrokerInterestLockBuffer.lockedAmount remains unchanged, and currentLocked() continues to subtract the pre-loss nominal amount from MoolahVault.totalAssets(). As a result, the loss is effectively reflected twice in the short term for the portion of the locked reward that was impaired: 1. the Moolah-side bad debt reduces raw vault assets;

2. the buffer continues subtracting the full pre-loss lock, including the part of the locked reward that no longer exists economically.

This does not necessarily create permanent insolvency if all LPs remain until the lock fully decays. The final raw vault assets will eventually become visible. The material issue is the interim ERC4626 pricing state: users who withdraw during the active lock can realize an overstated loss, while users who enter after the bad debt can participate in the later release of value that was over-suppressed.

Initial state:

- raw Moolah assets owned by the vault: 1,000
- active buffer lock: 100
- reported MoolahVault.totalAssets(): 900

A 10% bad-debt event affects the Moolah market holding those assets.

Raw Moolah assets become:

1,000 - 100 = 900

The buffer lock remains:

100

Actual reported totalAssets() becomes:

900 - 100 = 800

Expected proportional behavior if the locked reward were also haircut by the same 10% market loss:

- raw Moolah assets: 900
- surviving locked reward: 90
- expected reported totalAssets(): 810

The temporary NAV understatement is:

	810 - 800 = 10 This 10 corresponds to the impaired portion of the locked reward that the buffer continues to subtract even though it was already lost at the Moolah market level.
Recommendations	If such a situation happens, it can be partially resolved by decreasing the lock duration. However, there is no simple fix which prevents this issue from happening in the first instance.
Comments / Resolution	Acknowledged.

Issue_08	Griefing considerations due to non-zero currentLocked enforcement
Severity	**Low**
Description	The setLockBuffer function now checks that currentLocked is non-zero and only allows removing this in case it is zero. This will introduce a griefing vector as users could still relay interest to this buffer if the Broker -> Relayer -> Vault is part of the relayer system.
Recommendations	Consider removing the vault from the relayer system before the lock buffer is changed. Note that this will stop relay and corresponding repay/liquidate/.. actions.
Comments / Resolution	Acknowledged.

Issue_09	Baseline set during _setCap
Severity	**Low**
Description	MoolahVault tracks vault assets by iterating over markets in withdrawQueue inside totalAssets(). A market becomes part of that accounting surface when the curator calls setCap() with a positive cap for a market that is not currently enabled. In that branch, _setCap() pushes the market into withdrawQueue, sets config[id].enabled = true, and updates lastTotalAssets by adding the vault's existing MOOLAH.expectedSupplyAssets() balance for that market. lastTotalAssets is the vault's performance-fee baseline. _accrueFee() mints fee shares only on the positive difference between current smoothed totalAssets() and lastTotalAssets. The new broker relayers can supply broker revenue directly into Moolah on behalf of the vault by calling MOOLAH.supply(..., vault, ""). That path does not require the target market to be in the vault's current withdrawQueue, and it does not pass through the vault's _supplyMoolah() cap accounting. This creates a new interaction: a vault can already own Moolah supply shares in a market before that market is enabled in the vault accounting surface. The root cause is that _setCap() assumes existing vault assets in a newly enabled market should be treated as non-fee-bearing baseline assets. When newSupplyCap > 0 and config[id].enabled == false, _setCap() executes the following accounting transition: • it adds the market to withdrawQueue; • it marks the market enabled; • it increases lastTotalAssets by MOOLAH.expectedSupplyAssets(marketParams, address(this)). This is intended to avoid charging fees merely because a market with

	existing vault supply is added to the accounting surface. However, with the new relayer architecture, the “existing vault supply” can itself be broker yield or credit-broker recovery that was supplied into the market before the cap was enabled. Because _setCap() does not call _accrueFee() and does not distinguish pre-existing principal from newly generated broker revenue, that broker revenue is added directly to lastTotalAssets. Once added to the baseline, it is no longer treated as positive interest for vault fee accrual. The issue is reachable when a relayer is still able to supply into a broker market that is not currently enabled by the vault. The relayer checks broker token compatibility, but it does not check that the broker market is enabled, in withdrawQueue, in supplyQueue, or within vault cap. If the vault is whitelisted in Moolah for that market, the Moolah position can be created or increased before setCap() is called. Note that these scenarios can be sandwiched.
Recommendations	Consider acknowledging this issue. It appears very unlikely that relayers supply to a market that's not part of the queue, as this would suppress totalAssets temporarily.
Comments / Resolution	Acknowledged.

Issue_10	Incorrect fee mint calculation
Severity	Low
Description	MoolahVault charges performance fees by minting vault shares to feeRecipient. When _accrueFee() is reached, _accruedFeeShares() reads totalAssets(), computes the increase over lastTotalAssets, calculates feeAssets, and converts those fee assets into shares using _convertToSharesWithTotals(). The new broker-yield smoothing mechanism changes the meaning of totalAssets(). Broker revenue is first supplied into Moolah on behalf of the vault, so the vault immediately owns the full Moolah supply shares. The relayer then calls BrokerInterestLockBuffer.notifyBrokerInterest(amount), and MoolahVault.totalAssets() subtracts currentLocked() from the vault's raw counted Moolah assets. This means the vault's real backing already includes the full broker flush, but the user-facing ERC4626 NAV recognizes that value gradually. Fee accrual is performed against this smoothed NAV. The root cause is that _accruedFeeShares() prices fee shares using the smoothed newTotalAssets, while the minted shares are real vault shares that will participate in the later unlock of the remaining hidden broker yield. The relevant flow is: _accrueFee() calls _accruedFeeShares(). _accruedFeeShares() sets: newTotalAssets = totalAssets() After the buffer change, this is: raw vault Moolah assets - currentLocked() Then it computes:

totalInterest = newTotalAssets - lastTotalAssets

feeAssets = totalInterest * fee / WAD

and mints:

feeShares = _convertToSharesWithTotals(feeAssets, totalSupply(), newTotalAssets - feeAssets, Floor)

The denominator newTotalAssets - feeAssets is the smoothed total. It excludes the still-locked portion of the same broker flush, even though that portion is already held by the vault at the Moolah layer.

When the lock later decays, the remaining hidden value becomes visible in totalAssets(). The fee shares minted earlier remain outstanding and participate in that later reveal. Therefore, the fee recipient receives:

1. the intended fee value on the currently revealed tranche; plus
2. appreciation on those fee shares from later tranches of the same original broker flush.

The fee logic has no memory that the future totalAssets() increases are remaining pieces of an already-supplied broker reward. It treats each reveal as fresh interest and allows previously minted fee shares to participate in later reveals.

Without the buffer, the same broker flush would be recognized as a single lump, and the fee recipient's final claim would equal fee * X. With the buffer and intermediate fee checkpoints, the final claim exceeds fee * X.

Assume no virtual-share effect and no rounding for readability.

Initial vault state:

- Raw assets: 1,000
- Vault shares: 1,000
- lastTotalAssets = 1,000
- Performance fee: 10%

A broker flush supplies 100 assets into Moolah on behalf of the vault

and notifies the buffer.

Raw vault backing becomes:

1,100

But the full 100 is locked, so smoothed totalAssets() remains:

1,000

Expected single-lump fee without buffer

If the full 100 yield were recognized at once:

- feeAssets = 10
- fee recipient's final claim = 10

Actual two-step smoothed recognition

At half unlock, smoothed totalAssets() becomes 1,050.

- totalInterest = 50
- feeAssets = 5
- fee shares minted:

5 * 1,000 / (1,050 - 5) = 4.784689 shares

At full unlock, smoothed totalAssets() becomes 1,100.

The first fee shares are now part of totalSupply.

- totalInterest = 50
- feeAssets = 5
- second fee shares minted:

5 * 1,004.784689 / (1,100 - 5) = 4.588058 shares

Total fee shares:

9.372747

Final fee recipient asset claim:

	9.372747 / 1,009.372747 * 1,100 = 10.214286 Expected fee: 10 Actual fee value: 10.214286 Excess fee: 0.214286, or approximately 2.14% above the intended fee. This overcharge occurs even though the sum of feeAssets is still 10. The excess comes from the appreciation of fee shares minted during the partial reveal.
Recommendations	Consider documenting this newly introduced behavior. We do not recommend changing the fee calculation since the effect will be net-positive for the protocol.
Comments / Resolution	Acknowledged.

Issue_11	Yield on buffer accumulates immediately
Severity	**Low**
Description	Whenever interest is provided and buffered, it is immediately supplied into the corresponding market on behalf of the vault. This means that any supplied interest is directly subject to yield which increases totalAssets even when the principal is not yet fully released.
Recommendations	Consider keeping that in mind.
Comments / Resolution	Acknowledged.

Issue_12	Buffer removal will result in totalAssets jump if buffer has leftover lock
Severity	**Low**
Description	Whenever a buffer is removed while it does not return currentLocked = 0, this will result in an immediate increase of totalAssets, as not the previous currentLocked value is not deducted anymore and totalAssets reflects all funds in all markets without any smoothing mechanism. This can also be abused by an attacker via sandwich-attack.
Recommendations	Consider keeping that in mind.
Comments / Resolution	Resolved. However, it is now code-enforced that there is no currentLocked value anymore. The idea behind the recommendation is to keep the code and only remove a market if there is no currentLocked value tied to it anymore. The code-based solution now exposes a griefing vector which is explained in a separate, new issue.

Issue_13	New depositors will capture yield from old depositors
Severity	Low
Description	As with every dripping/smoothing mechanism, the dripped yield is technically owed to the existing depositors at the time of notification. However, new users will also get a part of the yield if they deposit before the full yield has been released. This is a standard limitation in dripping-like systems.
Recommendations	Consider keeping that in mind.
Comments / Resolution	Acknowledged.

Issue_14	Fee change will accrue fee from already notified interest
Severity	Low
Description	Whenever governance changes the fee, it may be possible that there is currently buffered interest. This interest has already been notified but is only released over time, which means that the new fee value will now be applied to capture fee from that interest.
Recommendations	Consider keeping that in mind.
Comments / Resolution	Acknowledged.

Issue_15	Deflation of totalAssets in case of market removal while buffer is existing
Severity	Low
Description	Whenever a market is removed from the queue but there is still pending interest, it will result in a decrease of totalAssets because now the pending interest is not offset with an existing market that might hold value for the vault. In general, there are also other side-effects from removing markets.
Recommendations	Consider only ever removing markets if they are truly empty and do not hold any value.
Comments / Resolution	Acknowledged.

Issue_16	Deviation between locked amount and received amount
Severity	**Low**
Description	When a broker's interest is flushed, BrokerInterestRelayer.supplyToVault / CreditBrokerInterestRelayer.supplyToVault calls MOOLAH.supply(marketParams, amount, 0, vault, "") and then BrokerInterestLockBuffer.notifyBrokerInterest(amount). The supply increases the vault's position in the Moolah market, which MoolahVault.totalAssets() reads as ΣMOOLAH.expectedSupplyAssets(withdrawQueue[i], vault) – currentLocked(). The notify sets lockedAmount = currentLocked() + amount. And totalAssets() subtracts currentLocked() so the just-flushed interest is smoothed out and revealed as the lock decays. The intended invariant is that the supply (which raises the counted sum) and the lock (which lowers it) cancel at flush time, leaving totalAssets() flat. The buffer locks the nominal amount that was handed to MOOLAH.supply, but the amount actually credited to the vault's counted assets is rounded down twice: - Moolah.supply mints supply shares with toSharesDown - expectedSupplyAssets converts the vault's shares back with toAssetsDown. So the increase in the vault's counted expectedSupplyAssets is amount minus dust (a sub-share-worth shortfall), while currentLocked() rises by the full amount. The lock therefore over-subtracts by dust, and totalAssets() dips by that dust at flush time instead of staying exactly flat. A second, smaller term works the other way: currentLocked() itself is lockedAmount * (dur – elapsed) / dur, an integer division that truncates

	down, so on each combine-and-reset the re-locked residual is under-counted by ≤ 1 wei. Neither rounding is reconciled anywhere, notifyBrokerInterest never reads the realized expectedSupplyAssets delta, it trusts the relayer's nominal amount, but neither needs to be, because the gap is dust and disappears as the lock decays.
Recommendations	Consider keeping this in mind.
Comments / Resolution	Acknowledged.

Issue_17	Missing events for setName and setSymbol
Severity	**Informational**
Description	Both aforementioned functions do not expose events which makes it harder for third-parties to observe any important change.
Recommendations	Consider implementing events.
Comments / Resolution	Resolved.

Issue_18	Whitelisted addresses can distribute shares
Severity	**Informational**
Description	Currently, only whitelisted addresses can deposit/mint (if the whitelist is enabled). However, transfers are not subject to a whitelist which means that any whitelisted address can distribute shares freely. While this is most likely intended, it is worth noting.
Recommendations	Consider keeping that in mind.
Comments / Resolution	Acknowledged.

Issue_19	Inconsistent whitelist checks implemented in deposit/withdraw functionality
Severity	**Informational**
Description	The deposit and mint vault functions only check if the receiver of the shares is whitelisted. No checks are enforced on the msg.sender. Similarly, the redeem functionality implements no whitelist checks on both the msg.sender and receiver. This coupled with the fact that transfers from whitelisted to non-whitelisted addresses are not disallowed, enables complete bypass of the whitelist system.
Recommendations	Consider acknowledging this whitelist bypass loophole if intended. Otherwise implement whitelist checks on the necessary instances.
Comments / Resolution	Acknowledged.

Issue_20	Consider providing role admin flexibility instead of using hardcoded role admin
Severity	Informational
Description	Function setRoleAdmin is introduced to change the admin of a role to the DEFAULT_ADMIN_ROLE. In the context of the MoolahVault, this means setting the role admin of the role ALLOCATOR from MANAGER to DEFAULT_ADMIN_ROLE. However if the DEFAULT_ADMIN_ROLE now wants to assign the MANAGER or any other role as the role admin again, it cannot do so. *function setRoleAdmin(bytes32 role) external onlyRole(DEFAULT_ADMIN_ROLE) {* *_setRoleAdmin(role, DEFAULT_ADMIN_ROLE);* *}*
Recommendations	Consider allowing the DEFAULT_ADMIN_ROLE to set the role admin to any role instead of the hardcoded DEFAULT_ADMIN_ROLE.
Comments / Resolution	Acknowledged.

Issue_21	Performance fee is charged on credit-broker bad-debt principal recovery
Severity	Informational
Description	MoolahVault treats any increase in totalAssets() as yield and charges a performance fee on it (_accruedFeeShares(): totalInterest = totalAssets() - lastTotalAssets). When a CreditBroker bad-debt position is later repaid, CreditBroker._repay() (the isBadDebt branch) routes the recovered principal back to the vault through the normal relayer path (relayer.supplyToVault() → MOOLAH.supply(..., onBehalf = vault, "")). This raises totalAssets(), so the vault charges a performance fee on it. That recovered amount is lenders' own previously-lost principal being returned, not yield — so a fee should not be charged on it. The effect is amplified by the fact that the original loss was socialized to lenders with no offset, while the recovery is taxed, so lenders cannot fully recover even on a 100% repayment.
Recommendations	Consider acknowledging this issue, as routing the principal through a separate path risks introducing further issues.
Comments / Resolution	

BrokerInterestLockBuffer

The BrokerInterestLockBuffer contract is a simple storage implementation which is tied to a vault and keeps track of the current locked/buffered amount.

Whenever interest is routed via the CreditBroker or LendingBroker through the corresponding relayer contracts, this interest is used to deposit into Moolah on behalf of a vault, which then increases the shares which are entitled to that vault.

To counter JIT attacks, where users can deposit funds into the MoolahVault just before such a reward deposit and the get a proportional gain of the rewards, the BrokerInterestLockBuffer contract has been implemented as a solution which drips/buffers rewards over a specific period (between 1 hour and 3 days).

The logic simply decreases the totalAssets value by the return value of the currentLocked function which, at the time of reward notification, is equal to the amount of notified rewards.

Privileged Functions

- notifyBrokerInterest
- setDuration

Invariants

INV 1: notifyBrokerInterest should only ever be called by BrokerInterestRelayer and CreditBrokerInterestRelayer

Issue_22	Incorrect order of operations during interest notification prevents expected trigger of assets <-> totalAssets safeguard
Severity	Medium
Description	The contract implements the following safeguard: *// Cap the flush at vault.totalAssets() so a miswired notify cannot drive lockedAmount past* *// the vault's holdings and collapse NAV / enable an inflation attack.* *if (amount > IERC4626(vault).totalAssets()) revert AmountExceedsVaultAssets();* First of all, this safeguard is not really helpful because anyone can simply deposit assets on behalf of the vault which then would allow for an inflation attack. Secondly, this check is inaccurate because totalAssets already includes the amount value (the notification is only happening afterwards).
Recommendations	Since this code path exposes a more critical vulnerability, we recommend removing it.
Comments / Resolution	Resolved.

Issue_23	Relayer notifications will reset timing progress
Severity	Medium
Description	BrokerInterestLockBuffer smooths brokered interest by holding a lockedAmount that decays linearly to zero over duration from lastUpdate. MoolahVault.totalAssets() subtracts the live currentLocked(), so a freshly-flushed amount is hidden and revealed gradually. When a broker flush arrives, the relayer supplies the interest and calls notifyBrokerInterest(amount), which is meant to add the new amount to whatever is still locked. The lock's schedule (lastUpdate/duration) governs how quickly the hidden yield re-enters totalAssets(), which in turn drives LP share pricing, maxWithdraw, and fee accrual. The combined step does newLocked = currentLocked() + amount and then resets lastUpdate = block.timestamp for the whole combined balance. There is no minimum on amount and no mechanism that keeps the pre-existing residual decaying on its original clock. As a result, the entire still-unrevealed remainder of prior flushes (the "old yield") is captured into newLocked and re-locked to a brand-new full duration every time a notification occurs, even a 1-wei or zero notification. Each flush thus restarts the reveal of the old yield from a higher value, pushing its reveal back by up to duration. Because the relayer notifies on every interest flush and flushes can be small and frequent, the residual is repeatedly re-locked, so totalAssets() stays suppressed by ~one duration of yield for as long as flushing continues. Nothing prevents this: the function trusts the caller and treats every notify identically. - t0: a flush locks X = 100 with duration = 3 days; currentLocked = 100, decaying. - t0 + 1.5d: half has revealed; currentLocked = 50, and totalAssets() reflects +50 of the yield.

	- t0 + 1.5d: a small flush of amount = 1 arrives → newLocked = currentLocked() + 1 = 51, lastUpdate = now. - Expected (old yield keeps revealing): the remaining 50 reveals over the next 1.5d. - Actual: that 50 is re-locked and now reveals over a fresh 3d; its reveal is deferred by up to 1.5d, and repeated small flushes keep deferring it.
Recommendations	Consider making sure that the relayer notification only happens with reasonable amounts and frequencies.
Comments / Resolution	Acknowledged.

Issue_24	setDuration will carry over leftover amount over full new duration
Severity	**Low**
Description	Whenever the setDuration function is called, it will use the leftover amount and set it to lockedAmount. This means the leftover amount (which would have been distributed over the leftover period), will now be distributed over the new period which is possibly longer or shorter.
Recommendations	Consider only changing the duration if it is absolutely needed and keep in mind that this will reset the timer for the leftover amount.
Comments / Resolution	Acknowledged.

Issue_25	Majority share holder in MoolahVault can brick interest notifications by performing withdraw-deposit sandwich
Severity	Low
Description	Assume the following scenario: - Alice is a majority share holder in the MoolahVault - Broker relayers attempt to notify interest to the lock buffer contract - Alice observes this notification and frontrun redeems her shares, withdrawing enough totalAssets such that the amount is greater than it. - Interest notification reverts due to the check in the code snippet below. *if (amount > IERC4626(vault).totalAssets()) revert AmountExceedsVaultAssets();* **Note this issue assumes the check above has been implemented correctly.** Currently the check is incorrect since the amount is already included in totalAssets (RHS) due to supply occurring before the interest notification through relayers. This has been described in a separate issue.
Recommendations	Consider removing this check completely as there are other related issues for it.
Comments / Resolution	Resolved.

Issue_26	_gap variable is not required
Severity	**Informational**
Description	The contract currently exposes a _gap variable which is usually used if a contract is meant to be inherited and used as a proxy implementation. In the current architecture, the BrokerInterestLockBuffer contract is deployed as a standalone contract and not inherited. Thus, the _gap variable is redundant.
Recommendations	Consider removing the _gap variable only if the contract is indeed never meant to be inherited.
Comments / Resolution	Resolved.

Issue_27	Buffer is not used to offset loss socialization
Severity	**Informational**
Description	Moolah markets can experience bad debt which is then socialized among all participants and the effect on the vault is a decrease in totalAssets. Such a decrease is not offset with the buffer and therefore is immediately realized while the buffer keeps dripping yield afterwards. This means users can simply escape loss socialization via withdrawals.
Recommendations	This is just a notice and we don't recommend attempting to implement such an offset mechanism as it would structurally change the logic and increase more complexity. It serves as a precondition for other issues but direct other fixes are more isolated.
Comments / Resolution	Acknowledged.

Issue_28	currentLocked rounds down
Severity	**Informational**
Description	Currently, the currentLocked function rounds down the locked amount which is in favor of users as the totalAssets() implementation deducts less buffer: *return (uint256(lockedAmount) * (dur - elapsed)) / dur;*
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_29	Remove redundant zero duration check from currentLocked
Severity	**Informational**
Description	The function currentLocked checks if the duration is zero and returns early if true. However it is not possible to set the duration to zero since both the initialize and setDuration functions ensure it is greater than or equal to *MIN_DURATION = 1 hours*. *function currentLocked() public view override returns (uint256) {* *uint64 dur = duration;* *if (dur == 0) return 0;*
Recommendations	Consider removing the check. Otherwise acknowledge the issue and keep it as a sanity check.
Comments / Resolution	Resolved.

Issue_30	Return early in currentLocked if lockedAmount equals zero
Severity	**Informational**
Description	Function currentLocked does not return early with 0 when lockedAmount equals 0. Since the eventual multiplication would evaluate to 0, returning zero is safe. *function currentLocked() public view override returns (uint256) {* *uint64 dur = duration;* *if (dur == 0) return 0;* *uint256 elapsed = block.timestamp - lastUpdate;* *if (elapsed >= dur) return 0;* *return (uint256(lockedAmount) * (dur - elapsed)) / dur;* *}*
Recommendations	Consider returning zero when lockedAmount is zero.
Comments / Resolution	Resolved.